particular version of the application, which in turn makes the acceptance criteria harder to integrate into a robust automated build system.

In addition, this approach can also encourage business analysts or other less technical team members to create scenarios in isolation. When developers come to implement these scenarios, they inevitably need to "tweak" them to make them easier to automate, so the automated scenarios end up being slightly different from the ones the business analysts originally proposed. This whole workflow breaks down the collaboration aspect of BDD that's so essential to its success.

For these reasons, it's important to store your executable specifications in version control. In fact, automated acceptance criteria should be considered a form of source code and stored in the same source code repository as your application code.

12.1.3 You should be able to run the executable specifications from the command line

You also need to be able to execute your acceptance criteria from the command line, typically using a build script. Graphical interfaces are convenient, but build automation needs scripting. And build automation is at the heart of the continuous integration and delivery strategies we'll discuss in the rest of this chapter.

There are many build-scripting tools, and your choice will typically depend on the nature of your project. In the Java world, you might use Maven, Gradle, or Ant. For a JavaScript-based project, you could use Grunt or Gulp. In .NET, it might be MSBuild or NAnt, and so on. In many cases, you could also launch the tests directly with the tool you're using (for example, using Cucumber with Ruby), although build scripts often give you more control over the execution environment.

All of the tools we've discussed in this book can be executed on the command line, or from within a build script, but many organizations still use more heavyweight test automation tools that integrate poorly with command-line execution and build automation. Without this capability, it becomes very hard to integrate the automated acceptance criteria into the automated build process, which in turn makes continuous integration and continuous delivery strategies very hard to implement.

12.2 Continuous integration speeds up the feedback cycle

One of the most important principles underlying agile and lean software development practices is feedback. Fast feedback loops are essential to reducing wasted effort and delivering valuable features efficiently. Being informed of an issue is the first step in resolving it. Delay represents risk: the faster you know about a potential problem, the better position you're in to adjust your actions accordingly. Even positive feedback is useful; when you can confirm that a particular strategy or problem solution is successful, you can build on this for subsequent work.

TDD or low-level BDD, for example, provides feedback within seconds or minutes, whereas automated acceptance tests, which typically take longer to run, can provide feedback within minutes or tens of minutes. In a project that relies entirely on manual